Lab 3C: CI/CD - Claude in Your Pipeline

Duration: 35 min

After: Production and CI/CD lecture (Day 3, Block 3)

Prerequisite: Lab 3B complete. Claude Code is installed and already signed in on this workstation. No Azure DevOps account or live pipeline access is required.

Objective

Run ordinary authenticated `claude -p` commands from the VS Code integrated terminal, restrict the available tools, bound the run, capture structured JSON, prove same-commit idempotency, and package the same review contract as a static Azure Pipelines design for an authorized owner.

Deliverable (toolkit chain 11 of 12)

- `scripts/ci-review.sh`
- `labs/3c-envelope.json`
- `reports/ci-review-policy.md`
- `azure-pipelines.yml`
- `labs/3c-azure-readiness.md`

START MARKER: *Open the VS Code integrated terminal in* `business-dashboard`*. Use the same workstation login that already works in Claude Code. Do not use* `--bare` *and do not create or paste an API key.*

Phase 1: Authenticated Print Mode (20 min)

Print mode means one prompt in, output out, and then the command exits. It uses the existing Claude Code login on this workstation.

Step 1: Prove the existing login works

Run this exactly in the VS Code terminal:

```
claude -p "What is the capital of France? Reply with only the city name."
```

Expected output marker: `Paris`, followed by a return to the terminal prompt. This proves that `claude -p` can complete an ordinary request using the workstation's current Claude Code login. It does not ask Claude to assert or certify an authentication state. If Claude asks you to sign in, stop and use the same approved Claude Code login used during the rest of the course.

Step 2: Run a bounded read-only repository command

```
claude -p "List all JS files in src/ and describe each in one sentence." --strict-mcp-config
--disable-slash-commands --setting-sources user --tools "Read,Glob" --allowedTools
"Read,Glob" --permission-mode dontAsk --max-turns 5
```

Expected output marker: a file list prints and the terminal prompt returns.

The controls have separate jobs:

- Existing login supplies authentication. There is no `--bare` credential path in this lab.
- `--strict-mcp-config` prevents unapproved MCP servers from being added to the run.

- `--disable-slash-commands` removes skills and slash commands from this automation path.
- `--tools` is the hard availability boundary.
- `--allowedTools` pre-approves only the exposed read tools.
- `--permission-mode dontAsk` denies anything else instead of waiting for a person.
- `--max-turns` limits the agent loop.

Ordinary print mode can still see approved user configuration and uses the workstation's saved Claude Code identity. This is appropriate for today's local exercise. A hosted pipeline needs an authorized execution identity and owner review before deployment.

Step 3: Inspect structured JSON

Run this from a Git Bash terminal inside VS Code so the schema quoting is predictable:

```
SCHEMA='{"type":"object","properties":{"issues":{"type":"array"},"score":{"type":"number"}}
,"required":["issues","score"]}'
claude -p "Review src/routes/tasks.js for security and quality issues." \
--strict-mcp-config --disable-slash-commands --setting-sources user \
--tools "Read,Grep,Glob" --allowedTools "Read,Grep,Glob" \
--permission-mode dontAsk --max-turns 5 \
--json-schema "$SCHEMA" --output-format json > review.json

node -e 'const j=require("./review.json"); console.log(j.subtype);
console.log(JSON.stringify(j.structured_output, null, 2)); console.log(j.num_turns);'
```

Expected output marker: the envelope has a success subtype, a `structured_output` object, and a numeric `num_turns`. Check `.subtype` before parsing because an error envelope may not contain `structured_output`.

Step 4: Run the reusable review script

Copy `Starter-Files/Day3_Lab_C/scripts/ci-review.sh` from the learner packet into `scripts/ci-review.sh`. Do not retype wrapped shell lines from the PDF. **Stay in the same Git Bash terminal for Steps 4 and 5.** PowerShell's `$?` is a Boolean and will display `True` or `False`, not the script's numeric exit code.

The script uses the same authenticated `claude -p` path and adds:

- read-only tools
- `dontAsk`
- a turn bound
- structured output validation
- fail-safe exit handling
- a commit-SHA idempotency marker

Run:

```
chmod +x scripts/ci-review.sh
./scripts/ci-review.sh
mkdir -p labs
cp review.json labs/3c-envelope.json
```

Expected output marker: `Review score: <n>` and either `REVIEW PASSED` with exit 0 or `ADVISORY FINDING` with exit 1. Exit 2 means an execution, turn-bound, or envelope failure and must fail safe.

Step 5: Prove same-commit idempotency

```
./scripts/ci-review.sh
code=$?
echo "exit code: $code"
```

Expected output marker: `SKIP: HEAD <sha> already reviewed - reusing verdict <n>.` No second Claude call occurs.

Add local evidence files to `.gitignore`:

```
printf "\n.ci-review/\nreview.json\n" >> .gitignore
```

Phase 2: Build an Azure-Ready Review Package (15 min)

You are producing a static handoff for an authorized Azure DevOps owner. You are not claiming that Azure executed it.

Step 1: Add the review policy

Copy `Starter-Files/Day3_Lab_C/reports/ci-review-policy.md` to `reports/ci-review-policy.md` and confirm it defines:

- security, validation, reliability, testing, and maintainability criteria
- CRITICAL, HIGH, MEDIUM, and LOW severity
- an advisory first release
- human ownership of merge and release decisions

Step 2: Inspect the Azure Pipelines design

Copy `Starter-Files/Day3_Lab_C/azure-pipelines.yml` to the repository root.

The static sample assumes an **authorized self-hosted agent** where Claude Code is installed and authenticated before the job begins. It deliberately does not embed or request an API key. The Azure owner must validate the execution identity, licensing, organizational policy, runner isolation, and token refresh behavior before enabling Build Validation.

Step 3: Perform the static readiness review

Confirm these controls:

- [ ] The YAML requires an authorized self-hosted agent with Claude Code already authenticated
- [ ] No literal credential or API key appears in YAML, scripts, logs, or policy files
- [ ] Full Git history is available for the merge-base diff
- [ ] Review scope is limited to changed JavaScript files
- [ ] Restricted tools, `dontAsk`, model setting, output-token cap, and turn bound remain intact
- [ ] Exit 1 stays advisory; exit 2 fails safe
- [ ] Only complete exit-0/1 results receive the durable source-commit marker
- [ ] Comments identify Build Validation as a future authorized-owner step

Then run this prompt in Claude Code:

```
Review azure-pipelines.yml against scripts/ci-review.sh, reports/ci-review-policy.md, and
the eight-item readiness checklist in Lab 3C. Write labs/3c-azure-readiness.md with PASS or
REWORK for each item and file:line evidence. This is static review only. State clearly that
Azure DevOps did not execute it.
```

Expected output marker: `labs/3c-azure-readiness.md` contains eight evidence-backed decisions and says `Not executed in Azure DevOps`.

Step 4: Package the handoff

```
git add scripts/ci-review.sh reports/ci-review-policy.md azure-pipelines.yml
labs/3c-envelope.json labs/3c-azure-readiness.md .gitignore
git commit -m "feat: package bounded Azure advisory review"
```

The authorized Azure owner can later configure the self-hosted agent, verify its Claude Code identity, authorize the pipeline, configure Build Validation, and run the first advisory pull request.

Optional Challenge: Write the Advisory-to-Gating Decision (8 min)

Create `labs/3c-production-decision.md`:

```
# CI Review Production Decision
- Initial mode: Advisory
- Authorized execution identity owner:
- Self-hosted runner isolation reviewed by:
- Calibration period or sample size:
- Evidence required before gating:
- Findings that might eventually block:
- False-positive ceiling:
- Human override owner:
- Decision today: Keep advisory
```

Done when: every field has a defensible value and the decision remains advisory until a real Azure owner validates the execution identity and measured behavior.

VS Code Review Gate

Review the Source Control diff, Problems panel, local script output, JSON envelope, policy, Azure design, and static readiness report. The human decision is whether the package is ready for an Azure owner to test, not whether an unexecuted pipeline works.

FINISH MARKER: Success Checklist

- [ ] Ordinary authenticated `claude -p` worked from the VS Code terminal without `--bare`
- [ ] Read-only tools, `dontAsk`, and `--max-turns` bounded the run
- [ ] JSON envelope parsed and was saved to `labs/3c-envelope.json`
- [ ] `scripts/ci-review.sh` returns 0 for pass, 1 for advisory findings, and 2 for execution or envelope failure
- [ ] A same-commit rerun reused the stored verdict without another Claude call
- [ ] `reports/ci-review-policy.md` defines criteria, severities, and advisory rollout
- [ ] `azure-pipelines.yml` assumes an authorized authenticated self-hosted agent and contains no API key
- [ ] `labs/3c-azure-readiness.md` records eight static checks and says Azure did not execute it
- [ ] Source Control diff and local evidence were reviewed before the human decision

If Stuck

Problem	Recovery
`claude -p` asks for login	Open the terminal under the same workstation user and approved Claude Code installation used during class, then authenticate normally
Headless run cannot read files	Confirm `--tools` contains Read, Grep, and Glob and `--allowedTools` matches the intended read set
`review.json` has no `structured_output`	Inspect `.subtype` first; error envelopes may not have `structured_output`
Script always prints SKIP	Remove only the local marker for this exercise or commit a change so HEAD moves
Azure identity is uncertain	Mark the readiness item REWORK; the authorized Azure owner must validate the self-hosted execution identity before deployment
Azure DevOps is unavailable	Expected. The required outcome is the static handoff package, not a live run

Debrief Questions

1. What is the difference between `--tools` and `--allowedTools`?
2. Why does a turn-bound or invalid envelope exit non-zero?
3. Why is the Azure design advisory and unexecuted?
4. What must the Azure owner verify before relying on an authenticated self-hosted runner?